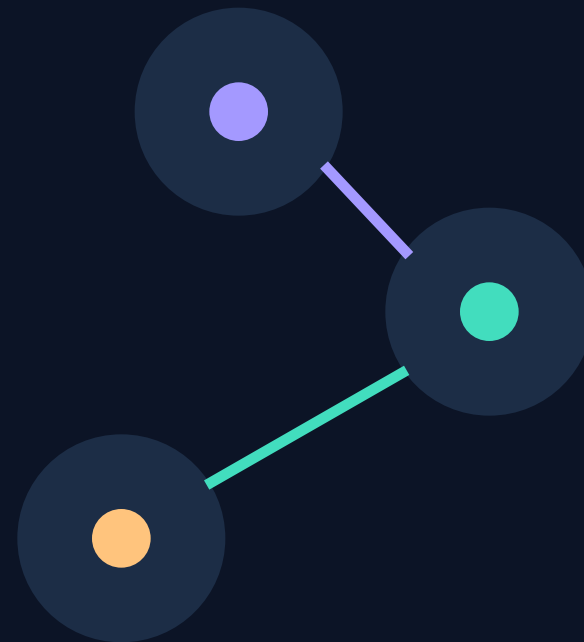


PLANNER + SCHEDULER

Let agents work in parallel. Not collide.

Evidence-aware planning and explainable, conflict-aware task batches for coding agents.



Parallel agents need coordination

More agents do not automatically mean faster delivery.

01

File collisions

Two agents edit the same source file. Merges and rework follow.

02

Wrong order

A dependent task starts before its prerequisite succeeds.

03

Repeated context

Related work reloads the same context and spends more compute.

Asynchronous work amplifies the cost of discovering conflicts late.

From issue to compatible batch

One workflow; distinct decisions for planning and scheduling.

01

Developer

Write a Markdown issue and declare target_files.



02

Planner

Refine steps, gather code evidence, request review.



03

Scheduler

Check state, priority, dependencies and file claims.



04

Proposal

Show selected tasks and exact reasons for deferral.

Humans assign the proposed work; the CLI does not start or reserve agents.

Protect the work already in motion

Every task is evaluated against the same input snapshot.

1 **Validate**

Positive capacity, unique IDs, known and acyclic dependencies.

2 **Gate**

Only pending work with already-succeeded prerequisites qualifies.

3 **Order**

Higher priority first; deterministic task-ID tie-break.

4 **Reserve**

Running tasks and selected tasks block overlapping file claims.

5 **Explain**

Return chosen IDs and a structured reason for each skip.

11 issues in; 4 selected

From examples/name-validation, scheduling the actual Markdown issue files.

SELECTED · CAPACITY 4

- 01 Normalize Unicode whitespace
- 02 Deduplicate batch names
- 03 Include total in summary
- 04 Accept CLI name input

EXPLAINED DEFERRALS

File conflict

Name length cap shares batch processor.

Dependency

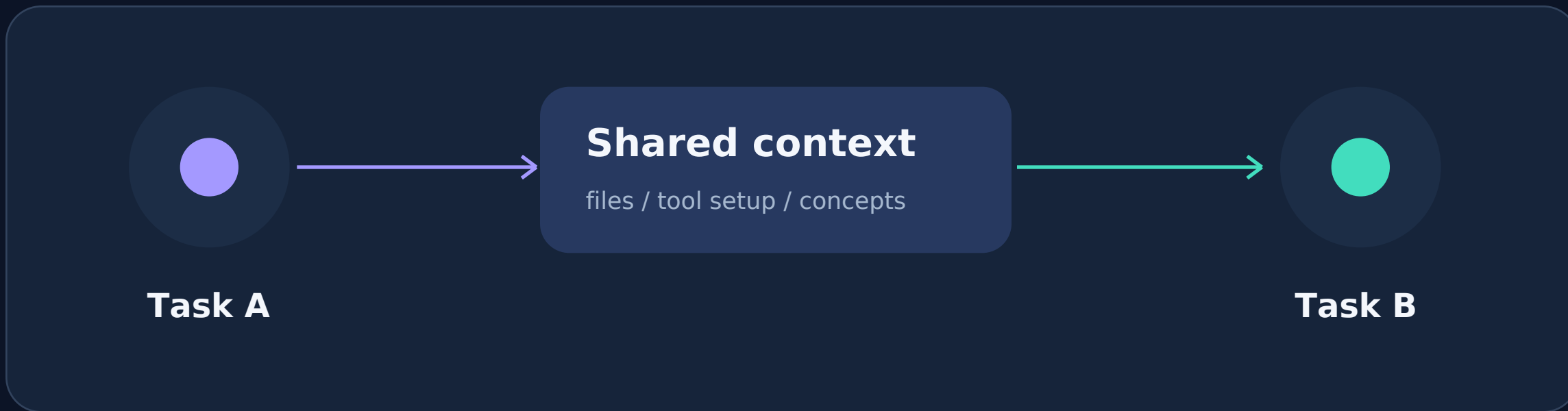
Document CLI input waits for CLI task.

Capacity

Batch ordering test waits for a slot.

Keep related work together

Affinity is an optional scheduling preference, not a cost measurement.



At the last available slot, an eligible member of an active affinity group may be preferred over unrelated work.

Potential benefit: less repeated context loading; actual savings need measurement.

Existing planner. Bob-built scheduler.

Bob implemented the scheduler as an independent Kotlin module.

EXISTING PLANNER FOUNDATION

Issue documents, evidence collection, readiness and review.

Configurable ACP connection can use an ACP-compatible IBM Bob agent for issue elaboration and review.

BOB-BUILT SCHEDULER

Pure batch selector, conflict explanations, tests and planner-app CLI integration.

Turns target_files into file reservations and proposes compatible work without executing tasks.

Run the real backlog demo

A planning proposal for parallel agents — with reasons you can inspect.

```
cd examples/name-validation
PLANNER=../../planner-app/build/install/planner-app/bin/planner-app
args=()
for issue in docs/internals/backlog/issue-*.md; do
    args+=(--issues "$issue")
done
"$PLANNER" schedule --capacity 4 "${args[@]}"
```

Build the CLI first: `./gradlew :planner-app:installDist`

PLAN CLEARLY. PARALLELIZE SAFELY. EXPLAIN EVERY DEFERRAL.